LSI POO 2026 - Trabajo Práctico Nº 5

Alumno/s: Caceres, Lourdes G. – Chávez, Tobías Acdel **DNI:** 45.940.137 – 47.616.254

Profesor: Mónica Ringa – Daniela Demchum

Grupo Laboratorio: 1 TP: 5 Fecha de entrega: 19/10/202

Introducción

El objetivo del Trabajo Práctico 5 fue aplicar los conceptos de herencia y polimorfismo, mediante la reutilización de clases definidas en trabajos anteriores (Persona, Punto, CuentaCorriente, Empleado, entre otras), como su estructura y su comportamiento en lugar de volver a escribirlos.

Algunos ejercicios requerían generalizar las características comunes en una superclase o especializar el comportamiento en las subclases y tratar de manera uniforme objetos de distintas clases a través de colecciones.

Desarrollo

Para el desarrollo del TP5 resolvimos situaciones de distinta naturaleza, que podemos agrupar en tres ejes:

1. **Herencia y redefinición de métodos.** Partimos de la clase Persona para construir subclases más específicas: Alumno, Empleado y Profesor. Cada una hereda los datos personales y redefine el método mostrar(), reutilizando el de la superclase y agregando su información particular: las notas y el promedio en el Alumno, la antigüedad y el sueldo neto en el Empleado, y el título, los cargos y el sueldo total en el Profesor. A su vez, los cargos de un profesor (Cargo, SemiExclusivo y Exclusivo) forman otra jerarquía en la que cada subclase redefine mostrarCargo() para agregar sus horas de investigación o de extensión, y la clase Facultad emite la nómina y el detalle de sus profesores.
2. **Generalización y especialización de figuras geométricas.** La clase Rectangulo fue especializada en Cuadrado, que redefine nombreFigura() y caracteristicas(). En sentido inverso, a partir de la clase Circulo ya existente generalizamos la clase Elipse, dado que un círculo es una elipse con sus dos semiejes iguales: Elipse concentra los atributos y las operaciones comunes (superficie, desplazamiento, distancia entre figuras) y Circulo queda reducido a sus constructores y al nombre de la figura.
3. **Clases abstractas.** Para los artefactos del hogar definimos la clase abstracta ArtefactoHogar, que contiene los datos comunes (marca, precio y stock), el cálculo de la cuota de un crédito y un método abstracto, creditoConAdicional(), que Cocina, Heladera y Lavarropas implementan con su propia regla (seguro por compresor, descuento si el lavarropas no es automático). De manera análoga, la clase abstracta Alojamiento reúne lo común de Cabania y Hotel, que redefinen el cálculo del costo, la liquidación y el conteo de alquileres por tipo.

Teoría Aplicada

En el TP5 aplicamos de manera práctica los siguientes conceptos fundamentales de la Programación Orientada a Objetos:

- **Herencia:** una subclase hereda la estructura y el comportamiento de su superclase y solo define aquello que la diferencia. Lo aplicamos en las jerarquías de Persona, de Rectangulo y Cuadrado, y de Elipse y Circulo, invocando el constructor de la superclase con super(...) y evitando repetir atributos.

- **Redefinición y polimorfismo:** una subclase puede redefinir un método heredado, y la versión que se ejecuta depende de la clase real del objeto y no del tipo de la variable. En la clase Empresa, una única colección de Personas contiene personas y empleados, y el mensaje mostrar() ejecuta la versión de cada objeto; del mismo modo, Comercio recorre artefactos de distintas clases y Gerencia liquida cabañas y hoteles enviando el mismo mensaje liquidar().
- **Clases y métodos abstractos:** una clase abstracta representa un concepto general que no se instancia directamente (un artefacto o un alojamiento en general) y declara métodos sin implementación que establecen el protocolo que deben cumplir todas sus subclases concretas.
- **Reutilización de código:** gran parte del código partió de trabajos prácticos anteriores. Las clases Persona y Punto fueron la base de varias jerarquías, lo que evitó la duplicación de código y mantuvo un diseño más uniforme.

Conclusión

Este trabajo práctico nos permitió profundizar en los conceptos de herencia y polimorfismo, y comprobar cómo reducen la duplicación de código y facilitan la escalabilidad de los sistemas: por ejemplo, incorporar un nuevo tipo de artefacto, de alojamiento o de cualquier clase que requiera una nueva subclase, sin modificar el código que trabaja con la superclase.

Además, el uso de colecciones junto con el polimorfismo nos permitió gestionar conjuntos de objetos de distintas clases de forma uniforme. La generalización de lo común y la especialización de lo particular son la base para diseñar aplicaciones modulares y fáciles de mantener.